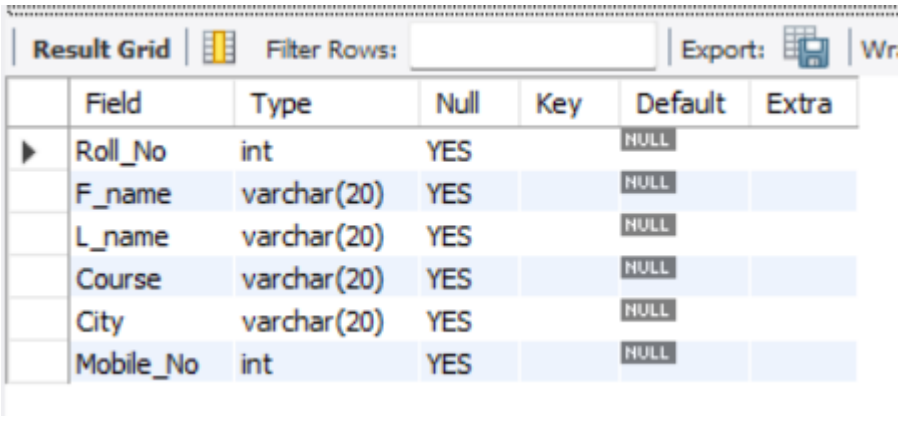
INDEX

Sr.no	Practical No	Date	
1.	To implement DDL (Data Definition Language) commands with suitable examples.		
2.	To implement DML (Data Manipulation Language) commands.		
3.	To implement various SQL functions- Number, Conversion, Date with examples		
4.	To implement character and conversion functions on the EMPLOYEE table.		
5.	To study and implement grouping and sorting in SQL		
6.	To create a database view for specific information retrieval.		
7.	To create and manage databases, tables, users, and roles.		
8.	To implement various SQL operators.		
9.	To implement Inner, Outer, and Natural Joins.		
10.	To solve problems using SQL functions on a library schema		
11.	To study stored procedures in PL/SQL without parameters.		
12	To create parameterized stored procedures in PL/SQL.		

Practical No. 1

- **Aim**: To implement DDL (Data Definition Language) commands with suitable examples.
- **Objective**: Learn to use `CREATE`, `ALTER`, `DROP`, and `TRUNCATE`.

Create : The CREATE TABLE statement is used to create a new table in a database
Query : create table student (Roll_No INT, F_name varchar(20) ,L_name varchar(20),Course varchar(20),City varchar(20), Mobile_No INT);
Ouput:-


Result Grid | Filter Rows: | Export: | Wr

Field	Type	Null	Key	Default	Extra
Roll_No	int	YES		NULL	
F_name	varchar(20)	YES		NULL	
L_name	varchar(20)	YES		NULL	
Course	varchar(20)	YES		NULL	
City	varchar(20)	YES		NULL	
Mobile_No	int	YES		NULL	

Alter: The ALTER TABLE statement is used to add, delete, or modify columns in an existing table.
The ALTER TABLE statement is also used to add and drop various constraints on an existing table.

ADD column:
Query : Alter table student ADD Email varchar(255);
Ouput:-

Field	Type	Null	Key	Default	Extra
Roll_No	int	YES		NULL	
F_name	varchar(20)	YES		NULL	
L_name	varchar(20)	YES		NULL	
Course	varchar(20)	YES		NULL	
City	varchar(20)	YES		NULL	
Mobile_No	int	YES		NULL	
Email	varchar(255)	YES		NULL	

DROP COLUMN:
Query : Alter table student DROP COLUMN Email;
Output:

Result Grid | Filter Rows: | Export:

Field	Type	Null	Key	Default	Extra
Roll_No	int	YES		NULL	
F_name	varchar(20)	YES		NULL	
L_name	varchar(20)	YES		NULL	
Course	varchar(20)	YES		NULL	
City	varchar(20)	YES		NULL	
Mobile_No	int	YES		NULL	

RENAME COLUMN:

Query : Alter table student Rename COLUMN Mobile_No to Phone_No;

Ouput:-

Field	Type	Null	Key	Default	Extra
Roll_No	int	YES		NULL	
F_name	varchar(20)	YES		NULL	
L_name	varchar(20)	YES		NULL	
Course	varchar(20)	YES		NULL	
City	varchar(20)	YES		NULL	
Phone_No	int	YES		NULL	

MODIFY DATATYPE :

Query : Alter table student MODIFY F_name varchar(40);

Output:-

Field	Type	Null	Key	Default	Extra
Roll_No	int	YES		NULL	
F_name	varchar(40)	YES		NULL	
L_name	varchar(20)	YES		NULL	
Course	varchar(20)	YES		NULL	
City	varchar(20)	YES		NULL	
Phone_No	int	YES		NULL	

Drop: The DROP TABLE statement is used to drop an existing table in a database.

Query : Drop Table student;

Output:-

17 21:29:10 Drop Table student 0 row(s) affected

Truncate : The TRUNCATE TABLE statement is used to delete the data inside a table, but not the table itself.

Query : Truncate table student;
Output:-

	Field	Type	Null	Key	Default	Extra
▶	Roll_No	int	YES		NULL	
	F_name	varchar(40)	YES		NULL	
	L_name	varchar(20)	YES		NULL	
	Course	varchar(20)	YES		NULL	
	City	varchar(20)	YES		NULL	
	Phone_No	int	YES		NULL	

YES

- **Conclusion**: DDL commands help define and manage database schema efficiently.

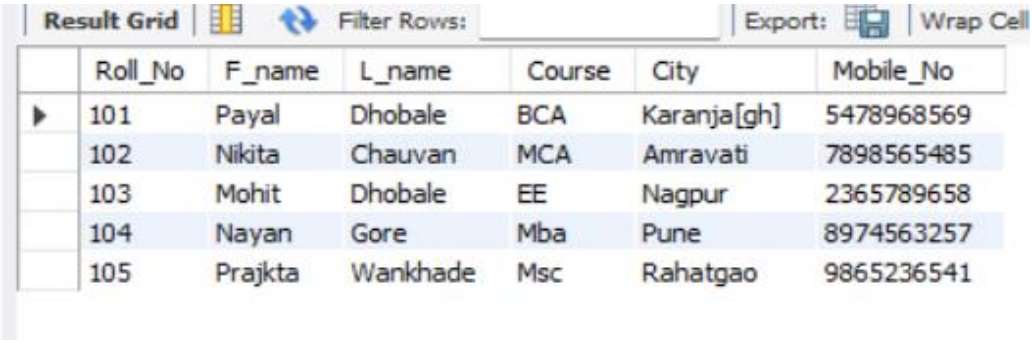
Practical No.2

- **Aim**: To implement DML (Data Manipulation Language) commands.
- **Objective**: Practice using `INSERT`, `UPDATE`, `DELETE`, `SELECT`.

Insert : The INSERT INTO statement is used to insert new records in a table.
Query:

```
Insert into students_info values(101,'Payal','Dhobale','BCA','Karanja[gh]',5478968569);
Insert into students_info values(102,'Nikita','Chauvan','MCA','Amravati',7898565485);
Insert into students_info values(103,'Mohit','Dhobale','EE','Nagpur',2365789658);
Insert into students_info values(104,'Nayan','Gore','Mba','Pune',8974563257);
Insert into students_info values(105,'Prajkta','Wankhade','Msc','Rahatgao',9865236541);
```

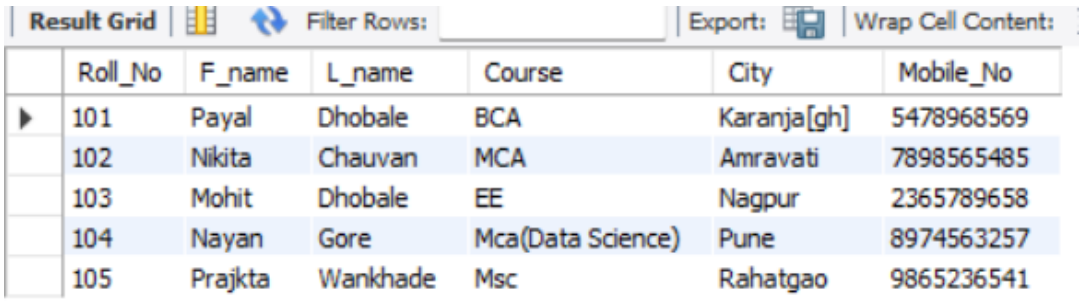
Output:



Result Grid | Filter Rows: | Export: | Wrap Cell

Roll_No	F_name	L_name	Course	City	Mobile_No
101	Payal	Dhobale	BCA	Karanja[gh]	5478968569
102	Nikita	Chauvan	MCA	Amravati	7898565485
103	Mohit	Dhobale	EE	Nagpur	2365789658
104	Nayan	Gore	Mba	Pune	8974563257
105	Prajkta	Wankhade	Msc	Rahatgao	9865236541

Update: The UPDATE statement is used to modify the existing records in a table.
Query: Update students_info set Course='Mca(Data Science)' where Roll_No=104;
Output:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Roll_No	F_name	L_name	Course	City	Mobile_No
101	Payal	Dhobale	BCA	Karanja[gh]	5478968569
102	Nikita	Chauvan	MCA	Amravati	7898565485
103	Mohit	Dhobale	EE	Nagpur	2365789658
104	Nayan	Gore	Mca(Data Science)	Pune	8974563257
105	Prajkta	Wankhade	Msc	Rahatgao	9865236541

Delete: The DELETE statement is used to delete existing records in a table.
Query: Delete from students_info where F_Name='Nayan';
Output:

Result Grid | Filter Rows: | Export: | Wrap Ce

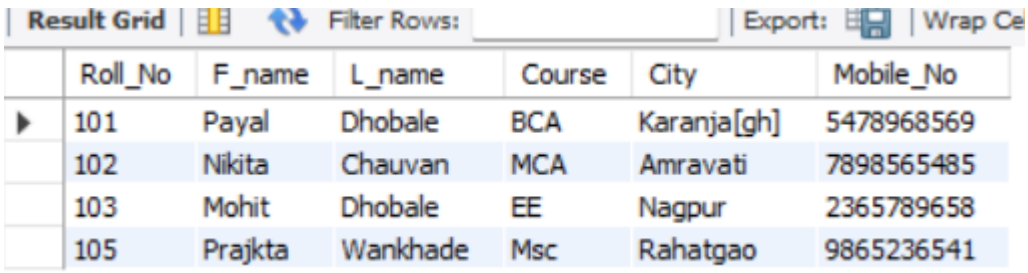
Roll_No	F_name	L_name	Course	City	Mobile_No
101	Payal	Dhobale	BCA	Karanja[gh]	5478968569
102	Nikita	Chauvan	MCA	Amravati	7898565485
103	Mohit	Dhobale	EE	Nagpur	2365789658
105	Prajkta	Wankhade	Msc	Rahatgao	9865236541

Select: The SELECT statement is used to select data from a database.

Query: Select* from students_info;

Select F_name, L_name , city from students_info;

Output:





- **Conclusion**: DML commands are essential for interacting with data in the database.

Practical NO. 3

- **Aim**: To implement various SQL functions- Number, Conversion, Date with examples.
- **Objective**: Understand how functions process data.

Number Function : functions include calculations like sum, average, max, min, rounding, and more.
Query: select * FROM EMPLOYEE3 WHERE Salary >(select max(salary) from EMPLOYEE3 where Designation IN ('Data Analyst','Software Engineer','Designation')) ;
Output:

Emp_no	E_name	E_address	E_ph_no	Dept_no	Dept_name	Job_id	Designation	Salary	Join_Date
101	Alice Johnson	New York, USA	9876543210	1	HR	201	MANAGER	75000	2015-06-15
106	Franklin Lee	Illinois, USA	9876543215	4	Operations	206	MANAGER	78000	2014-03-12
107	Grace Hopper	Virginia, USA	9876543216	2	IT	207	DEVELOPER	67000	2021-07-01
109	Ian Curtis	Nevada, USA	9876543218	6	Marketing	209	MARKETING MANAGER	73000	2013-08-15

Conversion: Conversion functions are used to convert one data type to another. For example, converting a string to a number, or a date to a string.
Query: SELECT Emp_no, E_name, CONVERT(Join_Date, CHAR) AS JoinDate_Text FROM EMPLOYEE3;
Output:

Emp_no	E_name	JoinDate_Text
101	Alice Johnson	2015-06-15
102	Bob Smith	2018-09-10
103	Charlie Brown	2016-01-20
104	David Wilson	2017-05-25
105	Emma Watson	2019-11-30
106	Franklin Lee	2014-03-12
107	Grace Hopper	2021-07-01
108	Hannah White	2020-12-10
109	Ian Curtis	2013-08-15
110	Jane Doe	2015-04-18

Date: Date functions are used to perform operations on date and time values, such as extracting parts of dates, calculating differences, or formatting.
Query: SELECT * FROM EMPLOYEE3 WHERE YEAR(Join_Date) > 1981 ORDER BY Designation ASC;

Output:

Emp_no	E_name	E_address	E_ph_no	Dept_no	Dept_name	Job_id	Designation	Salary	Join_Date
103	Charlie Brown	Texas, USA	9876543212	3	Finance	203	ACCOUNTANT	60000	2016-01-20
105	Emma Watson	Washington, USA	9876543214	1	HR	205	Data Analyst	50000	2019-11-30
107	Grace Hopper	Virginia, USA	9876543216	2	IT	207	DEVELOPER	67000	2021-07-01
110	Jane Doe	Ohio, USA	9876543219	2	IT	210	IT PROFF	59000	2015-04-18
101	Alice Johnson	New York, USA	9876543210	1	HR	201	MANAGER	75000	2015-06-15
106	Franklin Lee	Illinois, USA	3215	4	Operations	206	MANAGER	78000	2014-03-12
109	Ian Curtis	Nevada, USA	9876543218	6	Marketing	209	MARKETING MANAGER	73000	2013-08-15
108	Hannah White	Michigan, USA	9876543217	5	Sales	208	SALES EXECUTIVE	52000	2020-12-10
102	Bob Smith	California, USA	9876543211	2	IT	202	Software Engineer	55000	2018-09-10
104	David Wilson	Florida, USA	9876543213	2	IT	204	Software Engineer	62000	2017-05-25

- **Conclusion**: SQL functions simplify data processing and transformation tasks.

Practical No. 4

Aim: To implement character and conversion functions on the EMPLOYEE table.
Objective: Explore text and data-type transformation functions.

Character Function: Character functions are used to manipulate or transform text data (e.g., names, designations).
Query: SELECT Emp_no, UPPER(E_name) AS Upper_Name FROM EMPLOYEE3**;**
Output:

Emp_no	Upper_Name
104	DAVID WILSON
105	EMMA WATSON
106	FRANKLIN LEE
107	GRACE HOPPER
108	HANNAH WHITE
109	IAN CURTIS
110	JANE DOE

Query: SELECT Emp_no, LOWER(Designation) AS Designation_Lower FROM EMPLOYEE3;
Output:

Emp_no	Designation_Lower
101	manager
102	software engineer
103	accountant
104	software engineer
105	data analyst
106	manager
107	developer
108	sales executive
109	marketing manager
110	it proff

Query: SELECT Emp_no, LOWER(Designation) AS Designation_Lower FROM EMPLOYEE3;
Output:

Emp_no	Designation_Lower
101	manager
102	software engineer
103	accountant
104	software engineer
105	data analyst
106	manager
107	developer
108	sales executive
109	marketing manager
110	it proff

Conversion : Conversion functions are used to convert one data type to another. For example, converting a string to a number, or a date to a string.
Query: SELECT Emp_no, E_name, CONVERT(Join_Date, CHAR) AS JoinDate_Text FROM EMPLOYEE3;
Output:

Emp_no	E_name	JoinDate_Text
101	Alice Johnson	2015-06-15
102	Bob Smith	2018-09-10
103	Charlie Brown	2016-01-20
104	David Wilson	2017-05-25
105	Emma Watson	2019-11-30
106	Franklin Lee	2014-03-12
107	Grace Hopper	2021-07-01
108	Hannah White	2020-12-10
109	Ian Curtis	2013-08-15
110	Jane Doe	2015-04-18

Date: Date functions are used to perform operations on date and time values, such as extracting parts of dates, calculating differences, or formatting.
Query: SELECT * FROM EMPLOYEE3 WHERE YEAR(Join_Date) > 1981 ORDER BY Designation ASC;

Output:

Emp_no	E_name	E_address	E_ph_no	Dept_no	Dept_name	Job_id	Designation	Salary	Join_Date
103	Charlie Brown	Texas, USA	9876543212	3	Finance	203	ACCOUNTANT	60000	2016-01-20
105	Emma Watson	Washington, USA	9876543214	1	HR	205	Data Analyst	50000	2019-11-30
107	Grace Hopper	Virginia, USA	9876543216	2	IT	207	DEVELOPER	67000	2021-07-01
110	Jane Doe	Ohio, USA	9876543219	2	IT	210	IT PROFF	59000	2015-04-18
101	Alice Johnson	New York, USA	9876543210	1	HR	201	MANAGER	75000	2015-06-15
106	Franklin Lee	Illinois, USA	43215	4	Operations	206	MANAGER	78000	2014-03-12
109	Ian Curtis	Nevada, USA	9876543218	6	Marketing	209	MARKETING MANAGER	73000	2013-08-15
108	Hannah White	Michigan, USA	9876543217	5	Sales	208	SALES EXECUTIVE	52000	2020-12-10
102	Bob Smith	California, USA	9876543211	2	IT	202	Software Engineer	55000	2018-09-10
104	David Wilson	Florida, USA	9876543213	2	IT	204	Software Engineer	62000	2017-05-25

- **Conclusion**: These functions help enhance and customize data presentation.

Practical NO. 5

- **Aim**: To study and implement grouping and sorting in SQL.
- **Objective**: Use `GROUP BY`, `HAVING`, and `ORDER BY` for data summarization.

Group By: The GROUP BY statement groups rows that have the same values into summary rows, like "find the number of customers in each country".
Query: SELECT COUNT(CustomerID), Country FROM Customers GROUP BY Country;
Output:

Number of Records: 21

Expr1000	Country
3	Argentina
2	Austria
2	Belgium
9	Brazil
3	Canada
2	Denmark
2	Finland
11	France

Having: The HAVING clause was added to SQL because the WHERE keyword cannot be used with aggregate functions.
Query: SELECT COUNT(CustomerID), Country FROM Customers GROUP BY Country HAVING COUNT(CustomerID) > 5;
Output:

Number of Records: 5

Expr1000	Country
9	Brazil
11	France
11	Germany
7	UK
13	USA

Order By: The ORDER BY keyword is used to sort the result-set in ascending or descending order.

Query: SELECT * FROM Customers
ORDER BY Country ASC, CustomerName DESC;

Output:

Number of Records: 91

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
64	Rancho grande	Sergio Gutiérrez	Av. del Libertador 900	Buenos Aires	1010	Argentina
54	Océano Atlántico Ltda.	Yvonne Moncada	Ing. Gustavo Moncada 8585 Piso 20-A	Buenos Aires	1010	Argentina
12	Cactus Comidas para llevar	Patricio Simpson	Cerrito 333	Buenos Aires	1010	Argentina
59	Piccolo und mehr	Georg Pipps	Geislweg 14	Salzburg	5020	Austria
20	Ernst Handel	Roland Mendel	Kirchgasse 6	Graz	8010	Austria
76	Suprêmes délices	Pascale Cartrain	Boulevard Tirou, 255	Charleroi	B-6000	Belgium
50	Maison Dewey	Catherine Dewey	Rue Joseph-Bens 532	Bruxelles	B-1180	Belgium
88	Wellington Importadora	Paula Parente	Rua do Mercado, 12	Resende	08737-363	Brazil

- **Conclusion**: These clauses allow complex data analysis through aggregation.

Practical No. 6

- **Aim**: To create a database view for specific information retrieval.
- **Objective**: Learn how to encapsulate complex queries using `CREATE VIEW`.

View: In SQL, a view is a virtual table based on the result-set of an SQL statement. A view contains rows and columns, just like a real table. The fields in a view are fields from one or more real tables in the database.

Create View : A view is created with the CREATE VIEW statement.
Query: CREATE VIEW v_EmployeeReport AS SELECT Name, Department, JobTitle FROM Employee6;
Output:

	Field	Type	Null	Key	Default	Extra
▶	Name	varchar(100)	YES		NULL	
	Department	varchar(50)	YES		NULL	
	JobTitle	varchar(50)	YES		NULL	

.
Insert
Query: INSERT INTO v_EmployeeReport (Name, Department, JobTitle)

VALUES ('Ravi Kumar', 'Sales', 'Sales Executive');

Output:

	Name	Department	JobTitle
▶	Alice Johnson	HR	HR Manager
	Bob Smith	IT	Software Developer
	Charlie Brown	Finance	Accountant
	David Wilson	IT	System Analyst
	Eva Martinez	Marketing	Marketing Executive
	Frank Taylor	IT	Network Engineer
	Grace Lee	HR	Recruiter
	Hannah Davis	Finance	Financial Analyst
	Ravi Kumar	Sales	Sales Executive

Update
Query: UPDATE v_EmployeeReport SET Department = 'Marketing'

WHERE Name = 'Ravi Kumar';

Output:

Name	Department	JobTitle
Alice Johnson	HR	HR Manager
Bob Smith	IT	Software Developer
Charlie Brown	Finance	Accountant
David Wilson	IT	System Analyst
Eva Martinez	Marketing	Marketing Executive
Frank Taylor	IT	Network Engineer
Grace Lee	HR	Recruiter
Hannah Davis	Finance	Financial Analyst
Ravi Kumar	Marketing	Sales Executive

- **Conclusion**: Views simplify query writing and enhance data security.

Practical No. 7

- **Aim**: To create and manage databases, tables, users, and roles.
- **Objective**: Practice using user management and privilege commands.

Create a new user
Query: CREATE USER Paayall identified By 'abc' ;
Output:

#2. Develop a query to grant all privileges to employees table.
Query: GRANT ALL PRIVILEGES ON employee7 TO Paayall;
Output:

emp_name	dept_name	job_title	salary
Aarav	IT	Software Engineer	60000
Sneha	HR	HR Executive	45000
Rohan	Finance	Accountant	50000
Meera	IT	System Analyst	65000
John	Marketing	Marketing Manager	70000
Anjali	Sales	Sales Executive	40000
Vikram	Operations	Operations Manager	72000
Pooja	IT	Software Engineer	58000

#3. Develop a query to grant some privileges to departments table.
Query :GRANT SELECT, INSERT ON department TO Paayall;
Output:

dept_name	job_title	location
IT	Software Engineer	Mumbai
HR	HR Executive	Delhi
Finance	Accountant	Chennai
IT	System Analyst	Bangalore
Marketing	Marketing Manager	Hyderabad
Sales	Sales Executive	Pune
Operations	Operations Manager	Ahmedabad

#4. Develop a query to revoke all privileges of employees table.
Query :REVOKE SELECT,INSERT,DELETE,UPDATE ON employee7 FROM Paayall;

#5. Develop a query to revoke some privileges of departments table."
Query: REVOKE INSERT ON department FROM Paayall;

- **Conclusion**: Database security and organization depend on structured user and role management.

Practical No. 8

- **Aim**: To implement various SQL operators.
- **Objective**: Understand arithmetic, logical, comparison, special, and set operators.

Arithmetic Operator:
Query: select emp_name,salary ,salary+salary*0.1 as increased_salary from employee8;
Output:

emp_name	salary	increased_salary
Alice	55000.00000	60500.000000
Bob	48000.00000	52800.000000
Charlie	25000.00000	27500.000000
David	60000.00000	66000.000000
Eva	32000.00000	35200.000000
Frank	45000.00000	49500.000000
Grace	37000.00000	40700.000000
Hannah	52000.00000	57200.000000
Ian	58000.00000	63800.000000
Jack	30000.00000	33000.000000

Logical Operator:
Query: select * from employee8 where department='IT' and salary>50000;
Output:

emp_id	emp_name	department	salary	age	gender	joining_date	city
1	Alice	IT	55000.00000	28	Female	2020-06-15	Mumbai
4	David	IT	60000.00000	30	Male	2019-07-19	Chennai
8	Hannah	IT	52000.00000	32	Female	2016-05-30	Pune

Comparison Operator:
Query: SELECT * FROM Employee8 WHERE salary >= 40000;
Output:

emp_id	emp_name	department	salary	age	gender	joining_date	city
1	Alice	IT	55000.00000	28	Female	2020-06-15	Mumbai
2	Bob	Finance	48000.00000	35	Male	2018-03-22	Delhi
4	David	IT	60000.00000	30	Male	2019-07-19	Chennai
6	Frank	Sales	45000.00000	50	Male	2005-12-12	Delhi
8	Hannah	IT	52000.00000	32	Female	2016-05-30	Pune
9	Ian	Marketing	58000.00000	27	Male	2022-08-08	Mumbai

Special Operator: (IN, BETWEEN, LIKE, IS NULL)

IN : select * from employee8 where city in ('Mumbai', 'Delhi', 'Pune');
Output:

emp_id	emp_name	department	salary	age	gender	joining_date	city
1	Alice	IT	55000.00000	28	Female	2020-06-15	Mumbai
2	Bob	Finance	48000.00000	35	Male	2018-03-22	Delhi
3	Charlie	HR	25000.00000	45	Male	2010-11-01	Pune
6	Frank	Sales	45000.00000	50	Male	2005-12-12	Delhi
8	Hannah	IT	52000.00000	32	Female	2016-05-30	Pune
9	Ian	Marketing	58000.00000	27	Male	2022-08-08	Mumbai

Between: select * from employee8 where salary between 30000 and 60000;
Output:

emp_id	emp_name	department	salary	age	gender	joining_date	city
1	Alice	IT	55000.00000	28	Female	2020-06-15	Mumbai
2	Bob	Finance	48000.00000	35	Male	2018-03-22	Delhi
4	David	IT	60000.00000	30	Male	2019-07-19	Chennai
5	Eva	HR	32000.00000	29	Female	2021-01-05	NULL
6	Frank	Sales	45000.00000	50	Male	2005-12-12	Delhi
7	Grace	Finance	37000.00000	41	Female	2012-09-17	NULL
8	Hannah	IT	52000.00000	32	Female	2016-05-30	Pune
9	Ian	Marketing	58000.00000	27	Male	2022-08-08	Mumbai
10	Jack	Operations	30000.00000	26	Male	2021-04-01	Hyderabad

Like: select * from employee8 where emp_name like 'A%';
Output:

emp_id	emp_name	department	salary	age	gender	joining_date	city
1	Alice	IT	55000.00000	28	Female	2020-06-15	Mumbai

IS Null: select * from employee8 where city is null;
Output:

emp_id	emp_name	department	salary	age	gender	joining_date	city
5	Eva	HR	32000.00000	29	Female	2021-01-05	NULL
7	Grace	Finance	37000.00000	41	Female	2012-09-17	NULL

Set Operator: (UNION, INTERSECT, EXCEPT or MINUS)

Union : select * from employee8 where department='IT'
UNION select * from employee8 where department='HR';

Output:

emp_id	emp_name	department	salary	age	gender	joining_date	city
1	Alice	IT	55000.00000	28	Female	2020-06-15	Mumbai
4	David	IT	60000.00000	30	Male	2019-07-19	Chennai
8	Hannah	IT	52000.00000	32	Female	2016-05-30	Pune
3	Charlie	HR	25	5	Male	2010-11-01	Pune
5	Eva	HR	32000.00000	29	Female	2021-01-05	NULL

- **Conclusion**: Operators help build powerful conditions and expressions in SQL.

Practical No. 9

- **Aim**: To implement Inner, Outer, and Natural Joins.
- **Objective**: Combine data from multiple tables logically.

Inner Join: The INNER JOIN keyword selects records that have matching values in both tables.

Query: SELECT Orders.OrderID, Customers.CustomerName
FROM Orders
INNER JOIN Customers ON Orders.CustomerID = Customers.CustomerID;

Output:

OrderID	CustomerName
10248	Wilman Kala
10249	Tradição Hipermercados
10250	Hanari Carnes
10251	Victuailles en stock
10252	Suprêmes délices
10253	Hanari Carnes
10254	Chop-suey Chinese
10255	Richter Supermarkt

Outer Join: The FULL OUTER JOIN command returns all rows when there is a match in either left table or right table.

Query: SELECT Customers.CustomerName, Orders.OrderID
FROM Customers
FULL OUTER JOIN Orders ON Customers.CustomerID=Orders.CustomerID
ORDER BY Customers.CustomerName;

Output:

CustomerName	OrderID
Alice	101
Bob	102
Charlie	NULL
NULL	103
NULL	104

Natural Join: A NATURAL JOIN is a type of join that automatically joins two tables based on all columns with the same name and compatible data types in both tables.
Query: SELECT CustomerName, OrderID FROM Customers
NATURAL JOIN Orders;
Output:

CustomerName	OrderID
Alice	101
Bob	102

- **Conclusion**: Joins enable data integration and relational analysis.

Practical No. 10

- **Aim**: To solve problems using SQL functions on a library schema.
- **Objective**: Apply theoretical knowledge to a real-world-like schema.

SQL Functions: MySQL has many built-in functions**.**

Count: The COUNT() function returns the number of rows that matches a specified criterion.
Query: SELECT COUNT(*) AS TotalBooks FROM Books;
Output:

TotalBooks
3

Average: The AVG() function returns the average value of a numeric column**.**
Query: SELECT AVG(Price) AS AveragePrice FROM Books;
Output:

AveragePrice
516.33

Retrieve details of all books in the library – id, title, name of publisher, authors, number of copies in each branch, etc
Query: SELECT
B.Book_id,B.Title,B.Publisher_Name,A.Author_Name,C.Branch_id,C.No_of_Copies
FROM BOOK B
JOIN BOOK_AUTHORS A ON B.Book_id = A.Book_id
JOIN BOOK_COPIES C ON B.Book_id = C.Book_id
JOIN LIBRARY_BRANCH L ON C.Branch_id = L.Branch_id;
Output:

Book_id	Title	Publisher_Name	Author_Name	Branch_id	No_of_Copies
101	Learn SQL	Reilly Media	John Smith	1	4
102	Java	Pearson	Anita Desai	1	2
107	Machine Learning	Reilly Media	Tom Mitchell	1	3
110	JavaScript Guide	McGraw Hill	Douglas Crockford	1	2
103	Python	McGraw Hill	Robert Martin	2	5
104	Data Structures	HarperCollins	David Thomas	2	3
108	Database Systems	Pearson	Andrew Tanenbaum	2	4
105	Algorithms	Penguin Books	Brian W. Kernighan	3	6
106	C Programming	Pearson	Dennis Ritchie	3	2
109	AI for Beginners	HarperCollins	Rajeev Agarwal	3	1

Get the particulars of borrowers who have borrowed more than 3 books, but from Jan 2017 to Jun 2017

Query: SELECT Card_No, COUNT(*) AS Books_Borrowed
FROM BOOK_LENDING
WHERE Date_Out BETWEEN '2017-01-01' AND '2017-06-30'
GROUP BY Card_No HAVING COUNT(*) > 3;

Output:

Card_No	Books_Borrowed
201	4

Delete a book in BOOK table. Update the contents of other tables to reflect this data manipulation operation.

Query: DELETE FROM BOOK_AUTHORS WHERE Book_id = 105;

Output:

Book_id	Author_Name
101	John Smith
102	Anita Desai
103	Robert Martin
104	David Thomas
106	Dennis Ritchie
107	Tom Mitchell
108	Andrew Tanenbaum
109	Rajeev Agarwal
110	Douglas Crockford

- **Conclusion**: Practical exercises solidify understanding of SQL functions in real use cases.

Practical No. 11

- **Aim**: To study stored procedures in PL/SQL without parameters.
- **Objective**: Learn how to encapsulate logic for reusable execution.

Query:

```
CREATE OR REPLACE PROCEDURE Show_Recent_Employees
IS
BEGIN
   DBMS_OUTPUT.PUT_LINE('Employees who joined after 2020:');

   FOR emp_rec IN (
      SELECT Emp_no, E_name, Join_Date
      FROM EMPLOYEE3
      WHERE Join_Date > TO_DATE('2020-12-31', 'YYYY-MM-DD')
   ) LOOP
      DBMS_OUTPUT.PUT_LINE(
         'Emp No: ' || emp_rec.Emp_no ||
         ', Name: ' || emp_rec.E_name ||
         ', Join Date: ' || TO_CHAR(emp_rec.Join_Date, 'DD-MON-YYYY')
      );
   END LOOP;
END;
/
```

Output:-

```
Employees who joined after 2020:
Emp No: 101, Name: Ravi Kumar, Join Date: 10-JAN-2021
Emp No: 105, Name: Meena Das, Join Date: 15-FEB-2022
```

- **Conclusion**: Stored procedures promote reusability and modular programming in databases.

Practical No .12

Aim: To create parameterized stored procedures in PL/SQL.

- **Objective**: Understand passing values and updating tables dynamically.

Query:

```
CREATE OR REPLACE PROCEDURE Insert_Employee(
   p_Emp_no      IN EMPLOYEE3.Emp_no%TYPE,
   p_E_name      IN EMPLOYEE3.E_name%TYPE,
   p_E_address   IN EMPLOYEE3.E_address%TYPE,
   p_E_ph_no     IN EMPLOYEE3.E_ph_no%TYPE,
   p_Dept_no     IN EMPLOYEE3.Dept_no%TYPE,
   p_Dept_name   IN EMPLOYEE3.Dept_name%TYPE,
   p_Job_id      IN EMPLOYEE3.Job_id%TYPE,
   p_Designation IN EMPLOYEE3.Designation%TYPE,
   p_Salary      IN EMPLOYEE3.Salary%TYPE,
   p_Join_Date   IN EMPLOYEE3.Join_Date%TYPE
)
IS
BEGIN
   INSERT INTO EMPLOYEE3 VALUES (
      p_Emp_no, p_E_name, p_E_address, p_E_ph_no,
      p_Dept_no, p_Dept_name, p_Job_id, p_Designation,
      p_Salary, p_Join_Date
   );
   DBMS_OUTPUT.PUT_LINE('Employee inserted successfully.');
END;
/

//To Execute
SET SERVEROUTPUT ON;

EXEC Insert_Employee(
   201, 'Amit Shah', 'Pune', '9999999999',
   20, 'Finance', 102, 'Accountant',
   45000, TO_DATE('2022-06-15', 'YYYY-MM-DD')
);
//To Verify
SELECT * FROM EMPLOYEE3 WHERE Emp_no = 201;
```

Output:

Emp_no	E_name	E_address	E_ph_no	Dept_no	Dept_name	Job_id	Designation	Salary	Join_Date
201	Amit Shah	Pune	9999999999	20	Finance	102	Accountant	45000	15-JUN-22

- **Conclusion**: Parameterized procedures enhance flexibility and control over data manipulation.